

DenseNet

Densely Connected Convolutional Networks

Architecture Deep-Dive · Transfer Learning on CIFAR-10 · Results & Comparison

Jack Lichwa · Hemant Kumar · Prithika Kandasamy

Machine Learning II · Seattle University · Spring 2026

AGENDA

01 History & Motivation

Three structural barriers in deep CNNs
AlexNet → VGGNet → ResNet → DenseNet

02 Architecture Deep-Dive

Dense blocks · Bottleneck math
Transition layers · 121-layer count

03 Performance

Two-phase fine-tuning
CIFAR-10 results · DenseNet-121 vs ResNet-18

04 Advantages & Trade-offs

Feature reuse · Gradient flow
Activation memory · Deployment fit

PART 01

History & Motivation

Why the deep learning community needed a fundamentally new architecture

THREE STRUCTURAL BARRIERS IN DEEP CNNs

1 Vanishing Gradients

$$\frac{\partial L}{\partial x^0} = \prod_i \left(\frac{\partial x_i}{\partial x^{i-1}} \right) \cdot \frac{\partial L}{\partial x^1} \rightarrow 0$$

When the gradient signal at each layer is a fraction less than one, multiplying those fractions together across many layers shrinks the overall signal to near-zero, causing the network's earliest layers to completely stop learning.

2 Information Bottleneck

Layer ℓ sees only $x_{\{\ell-1\}}$

A feature discarded at any layer is permanently lost.

3 Feature Redundancy → Parameter Explosion

ResNet-50: 25.6 M params

Without access to prior representations, deep layers relearn low-level patterns already captured by early layers. Networks compensate with massive channel widths, producing millions of redundant parameters.

These three limitations motivated DenseNet's core design: concatenation over addition.

THE EVOLUTIONARY PATH: 2012 → 2017

2012

AlexNet

Used 8 layers and ~60M parameters to prove depth improves performance but suffered from vanishing gradients and hardware constraints.

2014

VGGNet

Increased depth to 16–19 layers but created a massive parameter explosion (>100M) due to independent feature learning in fully-connected layers.

2015

ResNet

Enabled large depth (50–152 layers) with ~60M parameters by using skip connections to solve vanishing gradients, though it was losing fine details in narrow bottlenecks

2017

DenseNet

Maximizes feature reuse by concatenating all preceding layers, which enables extremely deep networks (e.g., DenseNet-264 with only ~33M parameters)

SOLVING DEEP CNN BARRIERS: CNN → RESNET → DENSENET

Plain CNN

Features are completely overwritten every layer

$$x_l = H_l(x_{l-1})$$

- ✗ Vanishing gradients
- ✗ Features overwritten
- ✗ Redundant params

ResNet (2015)

Residual skip connections preserve early features

$$y = F(x) + x$$

- ✗ Addition blends & dilutes early features
- ✗ Parameter redundancy remains

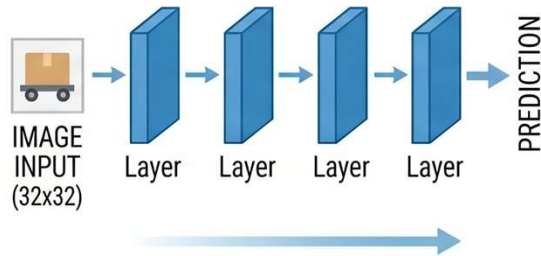
DenseNet (2017)

All prior feature maps concatenated as input

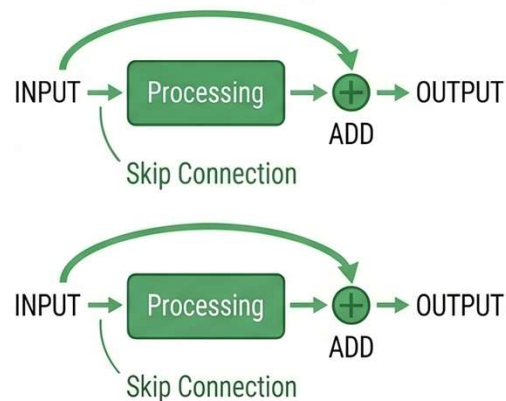
$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

- ✓ ∞ gradient paths → vanishing gradients impossible
- ✓ Full feature history preserved (append-only)
- ✓ 5–17× fewer params than ResNets

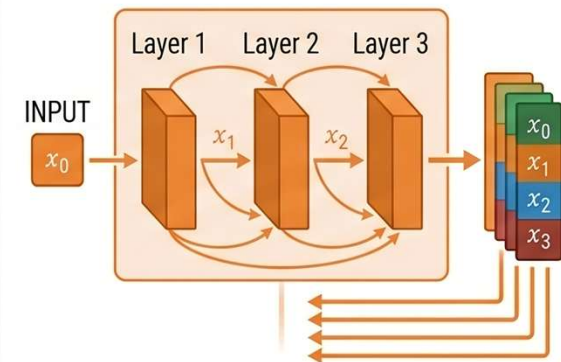
1. PLAIN CNN: SEQUENTIAL FLOW



2. RESNET: SKIP CONNECTIONS



3. DENSENET: FEATURE ACCUMULATION



Left: plain CNN overwrites. Centre: ResNet addition blends. Right: DenseNet concatenates — full history preserved.

DENSENET FAMILY: STANDARD IMAGENET VARIANTS

★ DenseNet-121 — Our Model

Dense layers per block: 6 / 12 / 24 / 16

Parameters: ~7.0 M (6.96 M trainable)

Memory (224×224): ~208 MB total

Best accuracy-per-parameter ratio. Ideal for transfer learning on modest GPU hardware.

DenseNet-169

6 / 12 / 32 / 32 · ~14.1 M params · Larger DB3 & DB4

DenseNet-201

6 / 12 / 48 / 32 · ~20.0 M params · DB3 is 2× DenseNet-121

DenseNet-264

6 / 12 / 64 / 48 · ~33.3 M params · Research / high-capacity

All variants share the same stem (7×7 Conv, stride 2 → 3×3 MaxPool), $\vartheta=0.5$ transitions, and classification head. Only dense layer counts differ.

PART 02

DenseNet-121 Architecture

Dense blocks · Bottleneck math · Transition layers · 121-layer count

DenseNet-121: Characteristics and Mechanics

121

Weight-bearing layers

121 Trainable Layers

- 116 Layers across 4 Dense Blocks
- 4 Layers in Transition Blocks (scaling down feature maps)
- 1 Layer at the input/output boundaries

~7M

Parameters

37% less than ResNet-18. Each layer only produces k new feature maps rather than wide activations in standard convnets.

$k = 32$

Growth rate

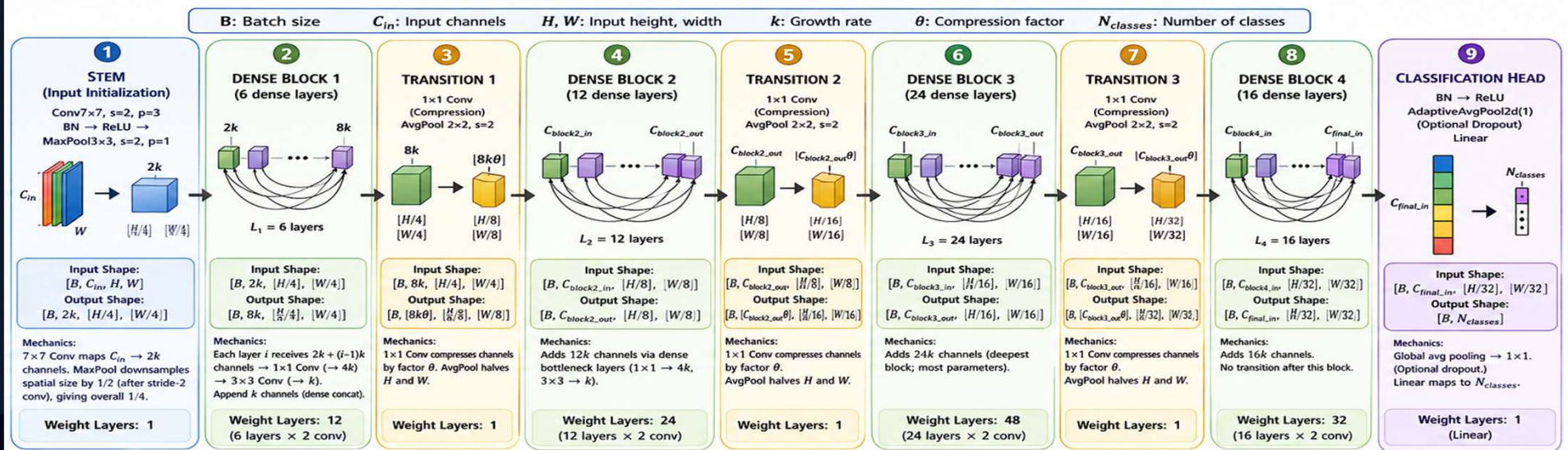
Every dense layer contributes exactly k new channels. Feature-map depth grows linearly, not exponentially.

DB3 ★

Dense Block 3 is the heavyweight

Holds ~70% of all parameters with 24 dense layers, each reading up to 288 input channels.

GENERIC DENSENET-121 BLOCK-BY-BLOCK MECHANICS



Nine-stage pipeline: input/output shapes, channel accumulation ($k=32, \theta=0.5$), weight layer counts. Dense Block 3 holds ~70% of all 7 M params.

DENSENET-121 HYPERPARAMETERS

Hyperparameters are fixed design decisions chosen before training. They govern how wide, how compressed, and how efficient the network is at every layer.

$k = 32$ Growth Rate

new channels per dense layer

Each dense layer adds **32 new feature channels** to the shared feature map.

Thus, layer ℓ receives $C_0 + 32(\ell - 1)$ input channels, enabling reuse of all previously learned features. A larger growth rate increases representational power but also **raises parameter and computational costs**.

$4k = 128$ Bottleneck Width

1×1 conv intermediate channels

The bottleneck layer uses a **1×1 convolution** to reduce the input to a fixed **128-channel** representation before the **3×3 convolution**.

This keeps the 3×3 parameter cost constant at **$128 \times k \times 3 \times 3$** , regardless of network depth.

$\theta = 0.5$ Compression Factor

transition layer channel reduction

Dense blocks accumulate channels rapidly. Every transition layer to **halve the channel count**, resetting the budget before the next block begins.

Channels climb inside each dense block, then drop sharply at the transition:

$64 \rightarrow 256 \rightarrow 128 \rightarrow 512 \rightarrow 256 \rightarrow 1024 \rightarrow 512 \rightarrow 1024$

Architecture Anchors — Stem & Head

Stem — Entry Point



The stem reduces input resolution before dense connectivity begins, since processing a **raw 224×224 image** through four feature-concatenating dense blocks would otherwise be prohibitively expensive in memory and FLOPs.

A **7×7 convolution (64 filters, stride 2)** transforms the raw pixel input into 64 feature maps while halving resolution to **112×112** ; **batch normalization** and **ReLU** then stabilize and activate those features. A **3×3 max-pool (stride 2)** applies a second spatial halving to **56×56** , a parameter-free operation that also introduces local translation invariance, before any dense layer processes the input.

Classification Head

Global Avg Pool $\rightarrow$ 1024-dim vector $\rightarrow$ FC $\rightarrow$ N classes

Dense Block 4 produces a **1024-channel feature volume** of spatial dimension **7×7** , where each channel encodes the spatial activation of one learned feature detector. **Global Average Pooling** reduces each channel's **7×7 map** to a single scalar by computing its spatial mean, collapsing **50,176 values to a 1024-dimensional vector** in which each element represents the global presence of one feature across the entire image.

A single fully-connected layer then projects those 1024 features into **N output logits** — one per class. **SoftMax** turns these into probabilities.

N

N is the only part that changes between tasks. The entire backbone — all four dense blocks — stays frozen. Set **$N = 1000$** for ImageNet, **$N = 10$** for CIFAR-10, **$N = 2$** for a binary tumor detector.

This single parameter swap is what makes DenseNet a universal backbone for transfer learning.

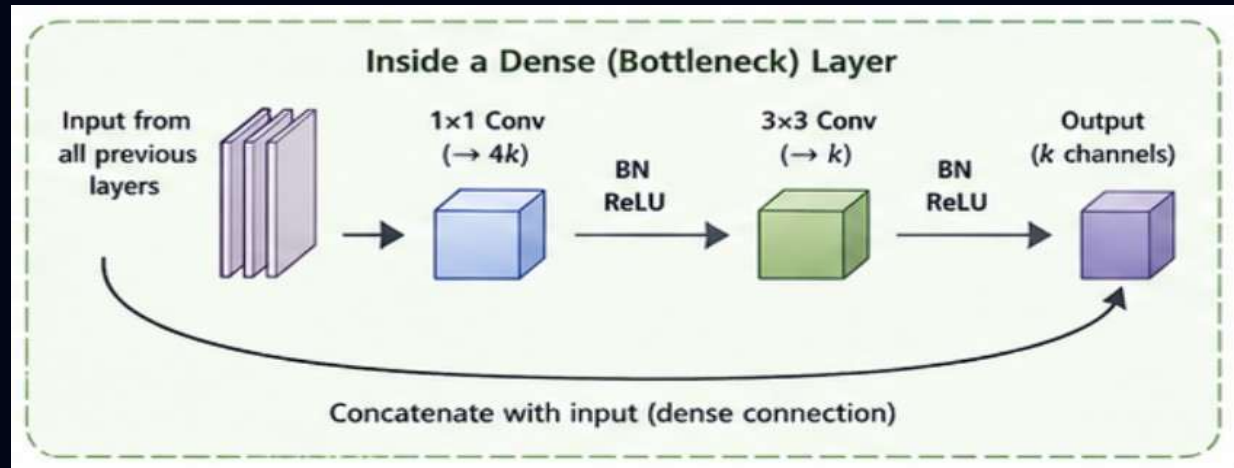
Bottleneck Layer Design: Taming the Growing Input

Two-stage design: compress then extract

$[C_{in}] \rightarrow 1 \times 1 \rightarrow 4k=128 \rightarrow 3 \times 3 \rightarrow k=32$

BN $\rightarrow$ ReLU applied before each convolution

Exactly 2 weight-bearing convolutions per dense layer



Dense Block 3 Example: the concatenated input is $C_0 + k \cdot (\ell - 1) = 256 + 32 \cdot 23 = 992$ channels.

Without Bottleneck

Direct 3×3 Conv on 992-channel input:

$$992 \times 32 \times 9 = 285,696 \text{ params}$$

With bottleneck

1×1 Conv first compresses to $4k = 128$ channels:

$$(992 \times 128) + (128 \times 32 \times 9) = 163,840 \text{ params}$$

TRANSITION LAYERS: BETWEEN-BLOCK COMPRESSION

Problem: Dense Blocks grow channels linearly. After DB3 the feature map reaches 1,024 channels at 14×14 — impractical without compression.

Two compressions ($\theta = 0.5$)

- *Spatial*: 2×2 Average Pooling
- *Channel*: 1×1 Conv $\rightarrow C_{out} = \lfloor C_{in} \times 0.5 \rfloor$

Stage-by-stage progression

Layer	Channels	Spatial
T1 (after DB1)	256 $\rightarrow$ 128	56×56 $\rightarrow$ 28×28
T2 (after DB2)	512 $\rightarrow$ 256	28×28 $\rightarrow$ 14×14
T3 (after DB3)	1024 $\rightarrow$ 512	14×14 $\rightarrow$ 7×7

No transition follows DB4 — spatial resolution stays at 7×7 until Global Average Pooling.



Top: channel count grows in Dense Blocks (blue) then halves at Transitions (orange, $\theta=0.5$). Bottom: spatial resolution drops 56 px $\rightarrow$ 7 px.

PART 04

Transfer Learning & Performance

Two-phase fine-tuning · CIFAR-10 results · DenseNet-121 vs ResNet-18

TWO-PHASE FINE-TUNING ON CIFAR-10

Phase 1 — Frozen Backbone (2 epochs)

What trains: Classifier head only — Dropout(0.3) + Linear(1024 → 10) · ~10,250 params (0.15% of total)

Learning rate: 1e-3 for head; backbone frozen

Purpose: Warm up the randomly-initialized head without corrupting ImageNet-pretrained backbone features. Converges quickly because only the tiny head updates.

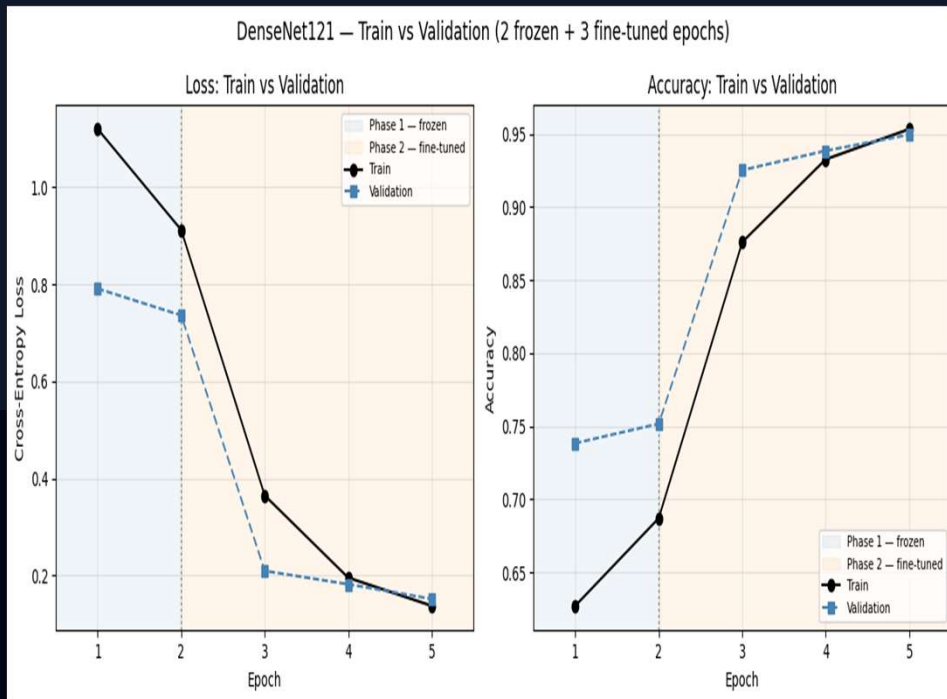
Phase 2 — Full Fine-Tune (3 epochs)

What trains: Full model — all 6,964,106 parameters

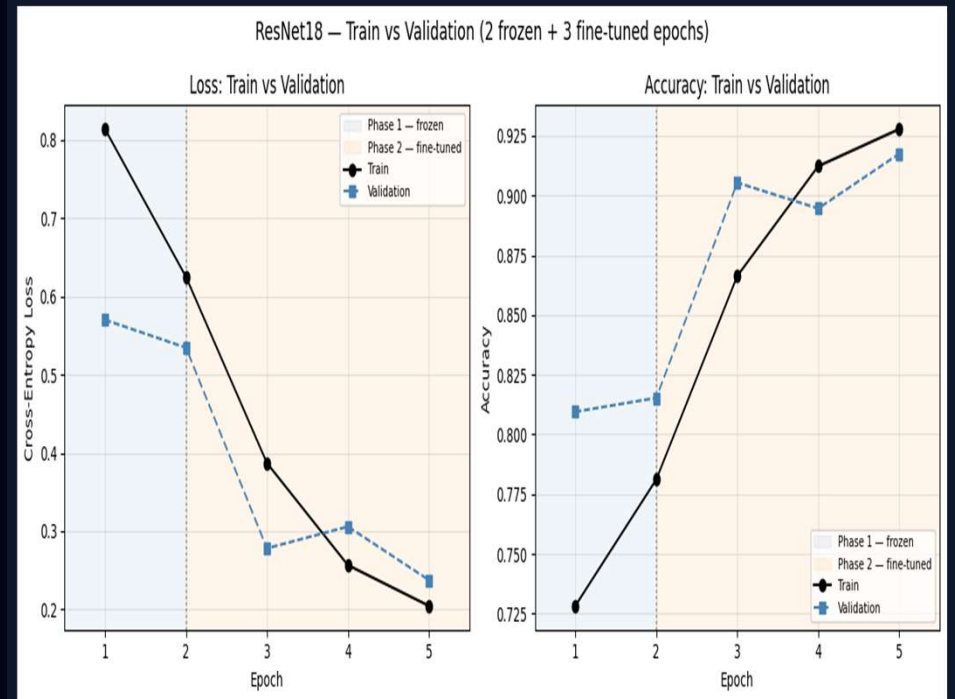
Learning rates: Backbone 5e-5 · Head 1e-3 (differential LR — 20× gap)

Purpose: Nudge the backbone toward CIFAR-10 at very low LR to prevent catastrophic forgetting. The 20× smaller backbone LR preserves pretrained low-level detectors.

TRAINING DYNAMICS: DENSENET-121 VS RESNET-18



*DenseNet-121: Phase 1 curves track closely (only 10K head params). Phase 2 sharp drop at epoch 3. Train $\approx$ Validation — no overfitting. **Final accuracy: $\sim 95.2\%$.***

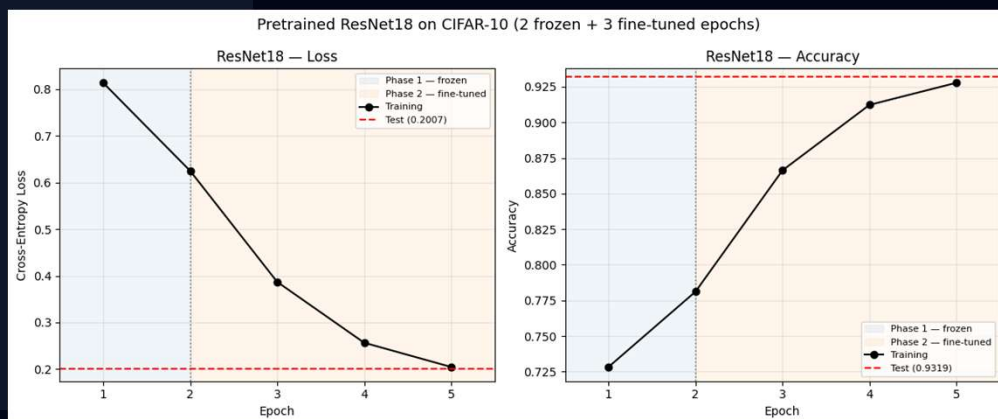


*ResNet-18: Same two-phase pattern. Phase 2 unfreeze at epoch 3 produces similar sharp drop. **Final accuracy $\approx 91.75\%$** — validation slightly above training (good generalization).*

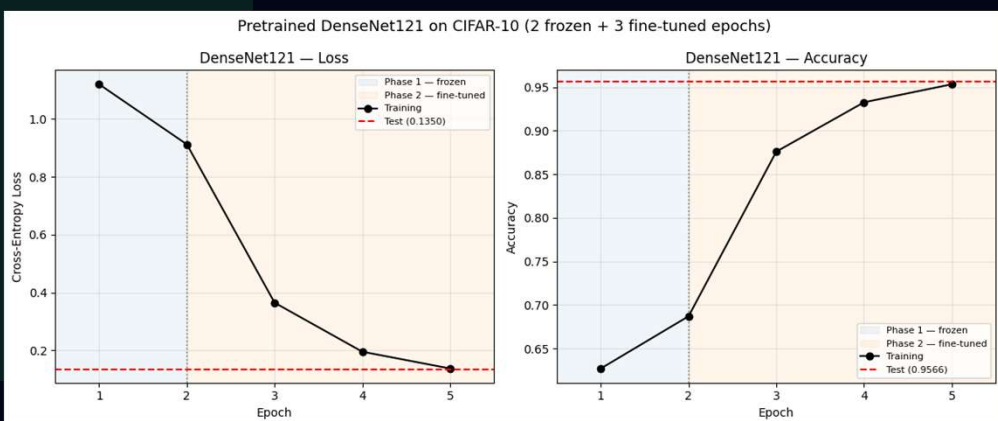
RESNET-18 VS DENSENET-121: TEST PERFORMANCE ON CIFAR-10

Side-by-side comparison of test loss and accuracy curves across 5 epochs of two-phase fine-tuning on CIFAR-10

ResNet-18



DenseNet-121



METRIC	DENSENET-121	RESNET-18	LAB 5 CNN
Test Accuracy	95.66%	93.19%	65.98%
Test Loss	0.135	0.201	0.970
Parameters	6.96 M	11.18 M	0.62 M

OVERALL

+2.47 pp accuracy gain

37% fewer parameters

Key result: DenseNet-121 achieves **+2.47 pp** over ResNet-18 using **37% fewer parameters**. The dashed red test line closely tracks the training curve — confirming minimal overfitting despite 7 M parameters fine-tuned on only 40 K training images.

Training dynamics: Sharp improvement at epoch 3 marks the Phase 2 backbone unfreeze. Both models converge by epoch 5, with DenseNet-121 showing stronger generalization.

DENSENET-121 VS RESNET-18:

Model comparison on CFAR-10 dataset

CRITERION	DENSENET-121	RESNET-18	WINNER
Test Accuracy (CIFAR-10)	95.66%	93.19%	DenseNet
Parameters	7.0 M	11.2 M (+60%)	DenseNet
GPU Memory / Image/64 Batch	~180 MB /~11 GB	~24.9 MB/~1.5 GB	ResNet
Test Loss (Final)	0.135	~0.201	DenseNet
Overfitting Risk	<0.5% gap	~1.1 % gap	DenseNet
Feature Preservation	Append-only concat, never overwrites	Additive blending	DenseNet
Edge / Mobile Readiness	Not ideal (memory-heavy, ~180 MB/image)	Smaller memory footprint	ResNet

✓ CHOOSE DENSENET-121 WHEN

- Accuracy and small storage is the **top priority**, not speed
- Model **feature reuse** matters (e.g., multi-task transfer)
- Training on **cloud GPU** with memory not constrained
- Working on **images or datasets** that benefit from dense features

⚡ CHOOSE RESNET-18 WHEN

- Real-time, **latency-sensitive**, or low-memory deployment
- Targeting **edge** device inference
- Constrained environment where **memory is limited** per batch

PART 03

Advantages & Trade-offs

Feature reuse · Gradient flow · Activation memory · Deployment fit

WHY DENSE CONNECTIVITY WORKS

① Explicit Feature Reuse

Every downstream layer reads all prior feature maps directly. Low-level edge detectors captured at layer 1 are still readable at layer 121 — no overwriting, no signal dilution.

② Hyper-Direct Gradient Flow

Layer ℓ has ℓ independent direct backward paths to the loss. Vanishing gradients are structurally impossible — there is always a short path for the gradient.

③ Parameter Efficiency

$k = 32$ is sufficient because layers never duplicate prior knowledge. DenseNet-121 beats ResNet-50 accuracy with 73% fewer parameters.

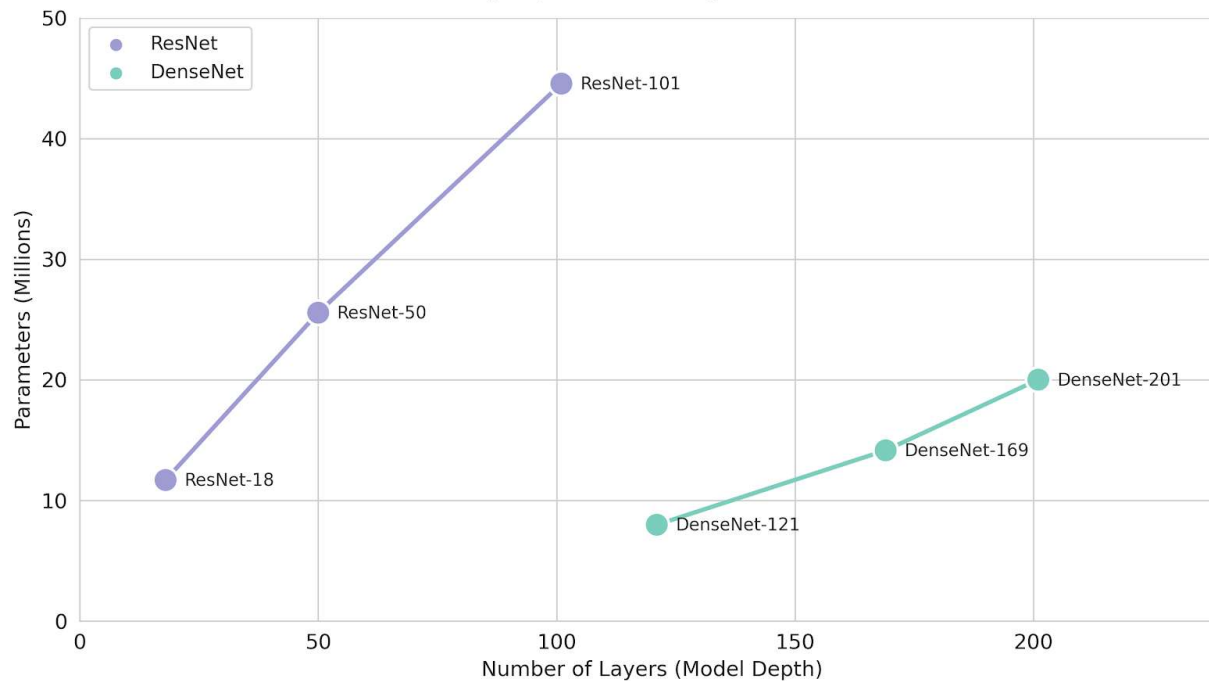
④ Complementary Learning (Implicit Regularization)

Since all prior representations are in the input, each new layer is forced to add genuinely new information — redundancy minimized by design.

DenseNet Parameter Efficiency vs Model Depth

Feature redundancy occurs when a network re-learns information already captured by earlier layers — wasting parameters on features that already exist rather than discovering new ones.

Parameter Efficiency by Model Depth (ResNet vs. DenseNet)



● ResNet

Residual shortcuts

Residual shortcuts let each layer add to its input. It leads to features from many layers back still require re-derivation, so parameters continue to grow steeply with depth.

● DenseNet

Full feature reuse

Every layer receives all prior feature maps directly via concatenation. No re-learning is needed — features computed at layer 1 remain explicitly available at layer 200.

84%

fewer params
vs. Plain CNN

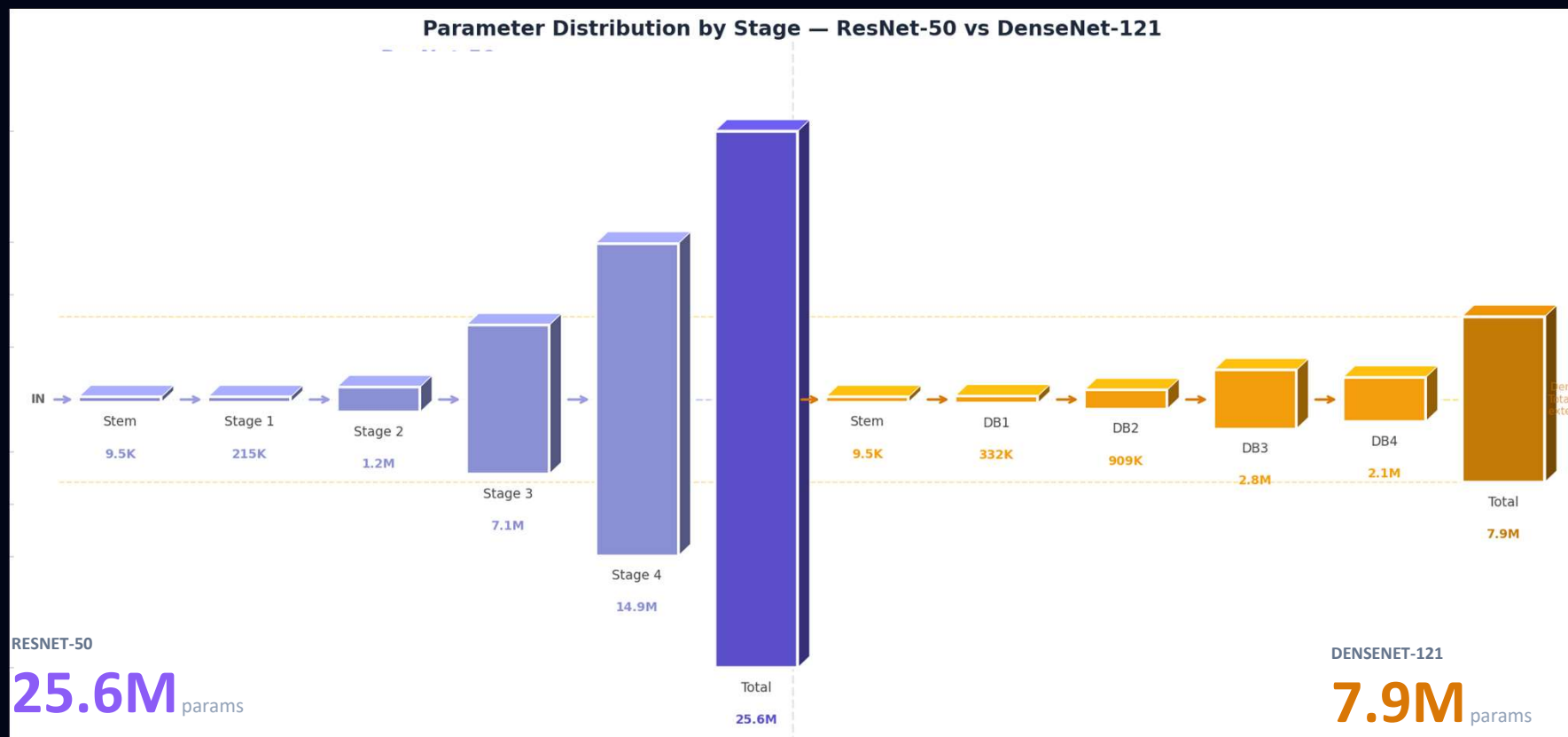
67%

fewer params
vs. ResNet

At 121 layers, DenseNet achieves **7-8 M parameters** vs. ResNet-101 44-45M — not by being shallower or narrower, but by **eliminating redundant re-learning entirely**. Dense feature reuse means every parameter is doing work that no other parameter has already done.

WIDE NETWORK VS NARROW LAYERS WITH FEATURE REUSE

ResNet parameters grow ~4x faster than DenseNet

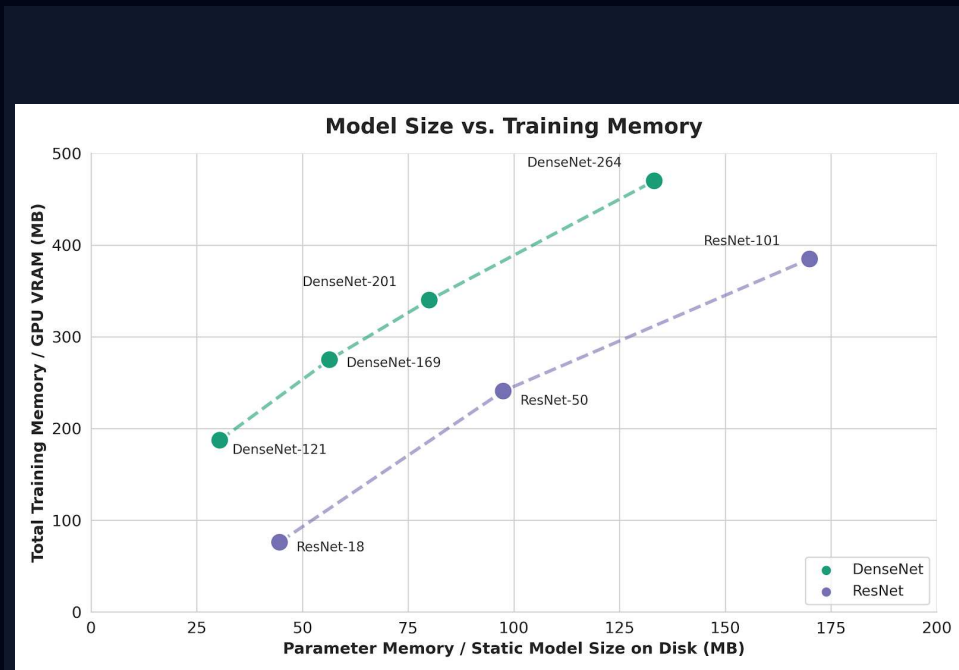


THE PRICE OF DENSE CONNECTIVITY: GPU MEMORY USAGE BY MODEL SIZE

DenseNet memory growth within dense blocks

In a DenseNet dense block with ℓ layers, each layer concatenates feature maps from all previous layers in the block. Therefore, all intermediate activations **must stay in GPU memory** until backpropagation through the entire block finishes. This causes **quadratic memory growth** — approximately $O(\ell^2)$ activation tensors (roughly $\ell(\ell+1)/2$ feature maps) per block.

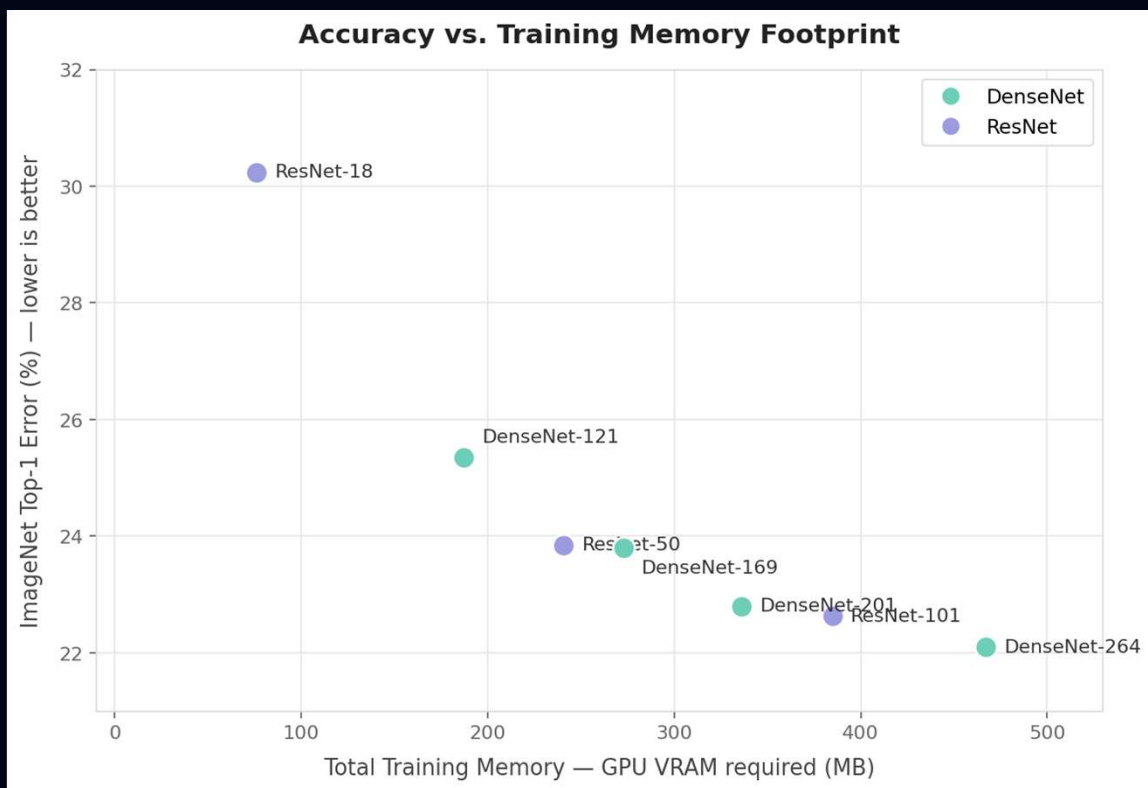
Model	Image Size	Memory per Image (Activation only)	64-Image Batch
Lab 5 CNN	32x32	0.46 MB	~29.4 MB
ResNet-18	224x224	~24.9 MB	~1.59 GB
DenseNet-121	224x224	~180 MB (~7×ResNet-18)	~11.5 GB



The chart shows activation memory usage (MB per image) during training for ResNet, and DenseNet at different depths. **DenseNet-264 (~135MB)** reaches ~480 MB per image, compared to ~390 MB for ResNet-101 (~165 MB)

DenseNet Accuracy Cost

Comparing accuracy and memory requirements across DenseNet and ResNet architecture variants.



- **Higher Depth**

~3× more layers

DenseNet-169 requires **~3× more layers** to achieve comparable accuracy to ResNet-50.

- **No Memory Gains**

Despite DenseNet's superior parameter efficiency, it requires roughly the **same memory footprint as ResNet to reach equivalent accuracy**.

- **Limited Production Use**

DenseNet offers **no reduction in memory footprint** (the primary cost and bottleneck in production pipelines) and suffers from slower training and inference.

As a result, DenseNet never gained widespread popularity.

DenseNet excels in niche use cases such as complex medical imaging and small datasets, where feature reuse is crucial.

KEY TAKEAWAYS

1. **Dense connectivity structurally solves vanishing gradients** — Every layer has direct paths to the loss, making gradient flow architecturally guaranteed within each block.
2. **Powerful feature reuse**— Concatenation preserves the full feature history from all previous layers, unlike ResNet's additive shortcuts that dilute early representations.
3. **Superior Parameters Efficiency** — DenseNet-121 reaches 95.66% accuracy on CIFAR-10 with 37% fewer parameters than ResNet-18 with 93.19% accuracy .
4. **Built-in Regularization Advantage** — The growth-then-narrowing design delivers lower overfitting risk (<0.5% gap) and more stable training compared to ResNet-18 (~1.1 pp gap)
5. **Memory Cost Is the Deciding Factor**— Despite DenseNet feature reuse and parameters efficiency by layer. It requires same number of parameters to achieve high accuracy in comparison to ResNet.
6. **Strong Specialized Use Cases**— DenseNet excels in medical imaging and small-dataset tasks, where fine feature reuse and reduced overfitting matter most.